Đoàn Việt Đức - 23020599

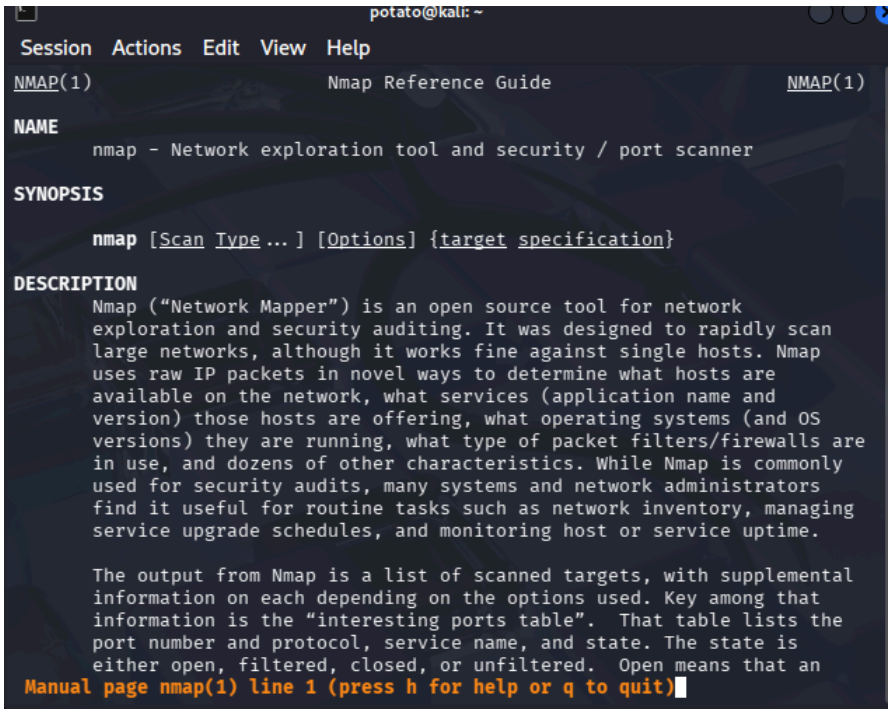
Lab Exercise 16.01: Port Scanning with Nmap

Step 1: The SYN scan starts off like a normal TCP three-way handshake, which establishes a connection between communicating machines. The client sends a TCP header with the SYN flag turned on. Nmap, by default, will scan the thousand most common ports, although you can specify a certain port or a custom range of ports. Each closed port in a SYN scan will respond by sending a TCP segment with the RST (reset) flag turned on. That's the way TCP/IP was designed. When closed ports receive a SYN, they reply with an RST, which immediately closes any connection or attempt to connect from a client

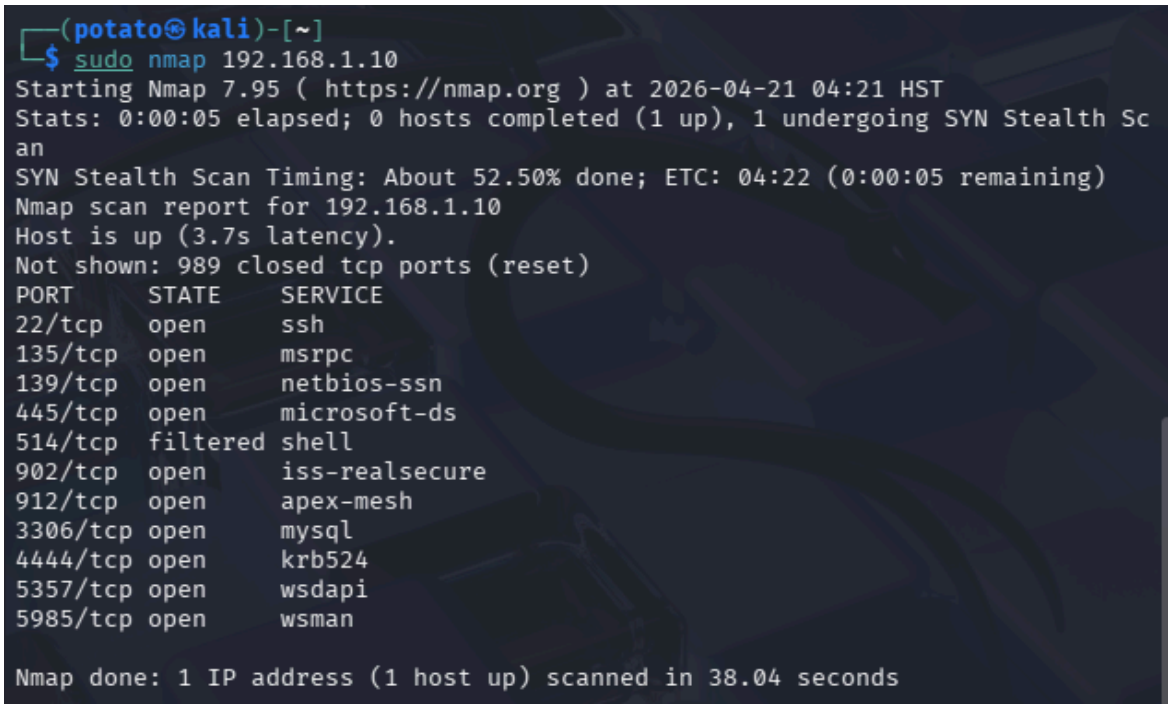
1a-1f

a) In the Kali Linux terminal, enter **nmap | less**

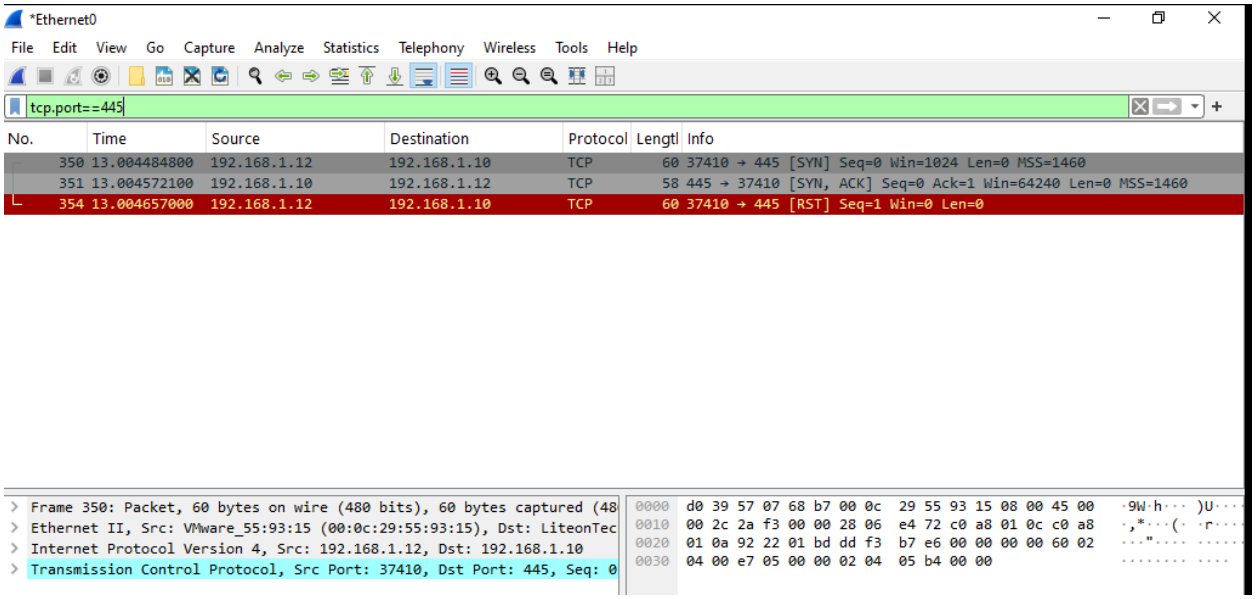
b) man nmap



c) sudo nmap 192.168.1.10 (WIN10 VM's IPv4 address)

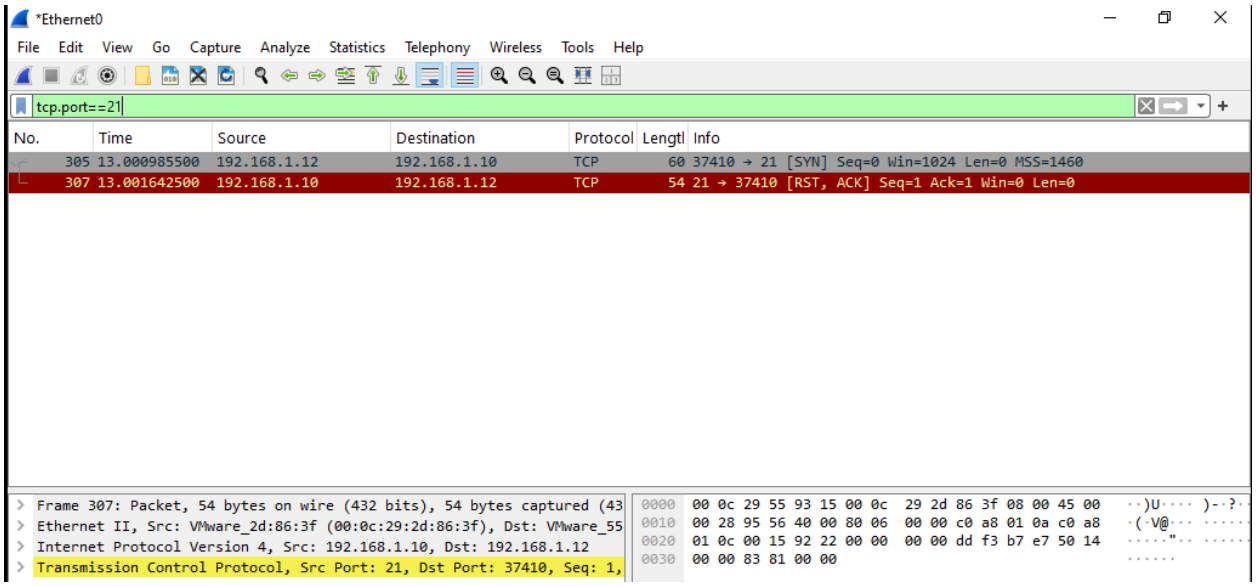


d-e) Change filter to **tcp.port==445**



-> After the Kali Linux VM sent the SYN, the open port 445 sent a SYN-ACK.Then the Kali Linux VM closed the connection with an RST.

f) Change filter to **tcp.port==21**



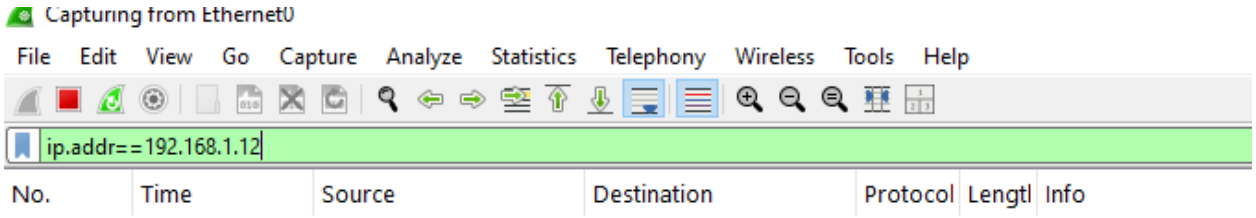
-> The Kali Linux VM sent the SYN, but **since port 21 is closed** (as there is no FTP server currently running on the Windows 10 VM), the Windows 10 machine responded back with an RST.

Step 2: A similar scan called the connect scan should almost never be used. The connect scan is named after the connect() function that operating systems use to initiate

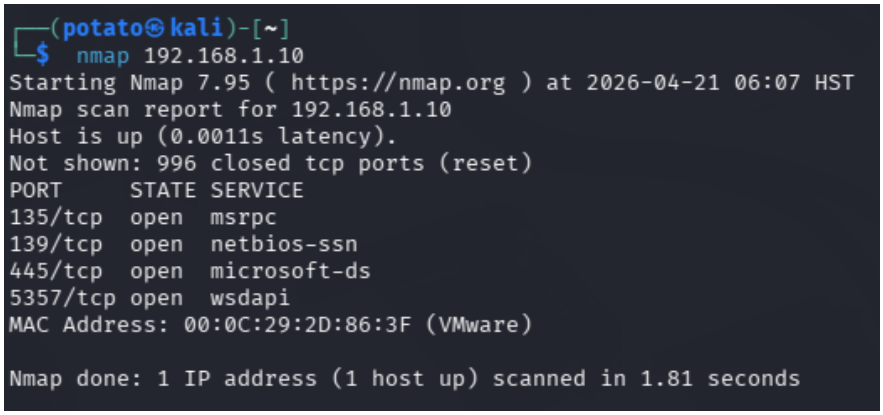
a TCP connection to another machine. This scan uses a normal TCP connection, the same method used by every TCP-based application, to determine if a port is available. This is not like the SYN scan, which uses Nmap to craft raw packets.

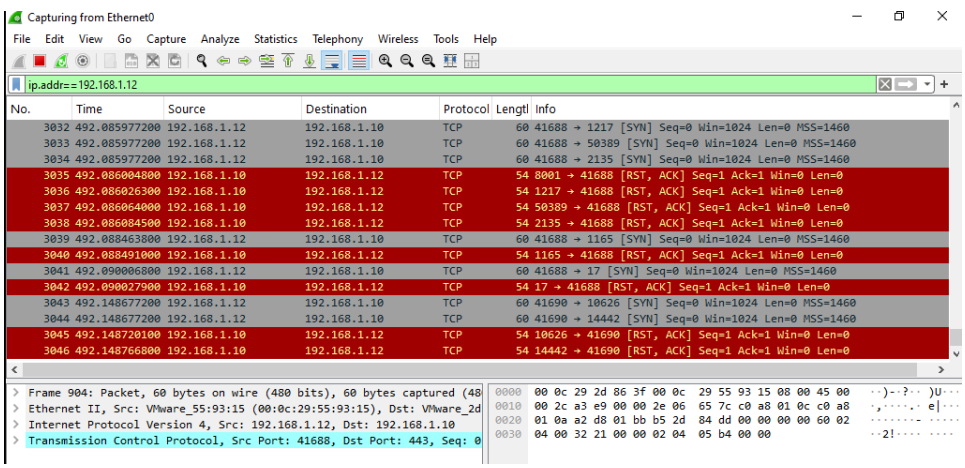
2a-2e

a) On the Windows 10 VM, start a new Wireshark capture by clicking the green fin on the toolbar (the third icon from the left). Once again, use a display filter of **ip.addr==192.168.1.114**, where you substitute the IP address of the Kali Linux VM following the ==:
- Start

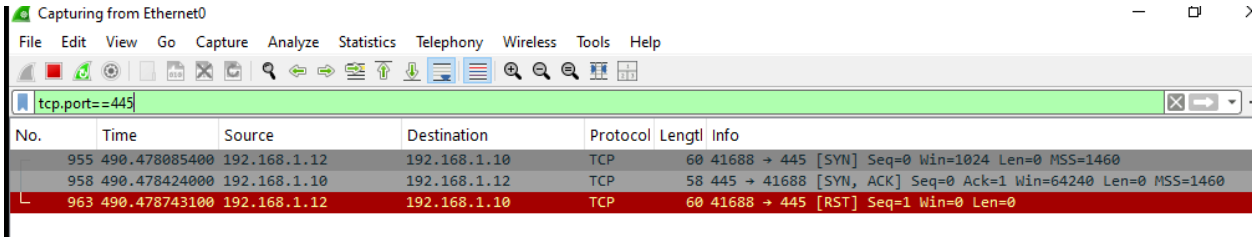


b) When the scan type is not specified (and sudo is not used), Nmap uses a connect scan: **nmap 192.168.1.10**



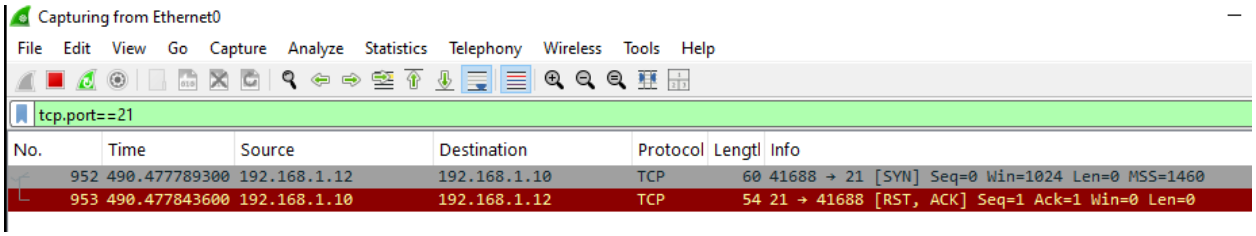


c-d) Change the filter to **tcp.port==445**



-> The connect scan identifies port 445 as open, just like the SYN scan did. However, with the connect scan, the TCP three-way handshake actually completes.

e) Change the filter to **tcp.port==21**

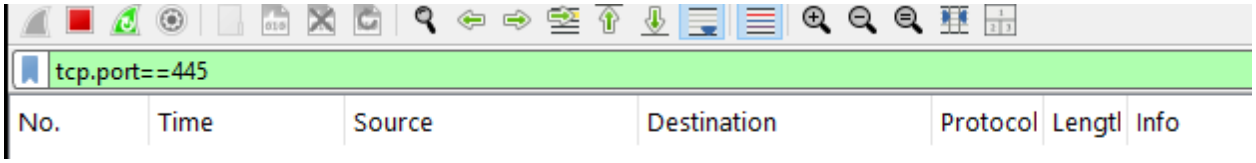


-> As with the SYN scan, with a connect scan, a closed port will respond to a SYN with an RST (which in this case is also accompanied by an ACK)

3a-3k

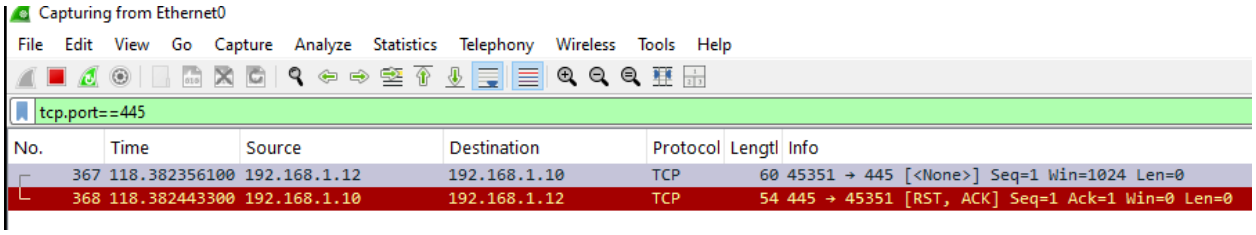
Step 3: Now, you'll execute Null, FIN, Xmas, and ACK scans

a) On the Windows 10 VM, start a new Wireshark capture - Use a display filter of **tcp.port==445**



b) On the Kali Linux VM, execute the Null scan with the following command:
sudo nmap -sN -p 445 192.168.1.10

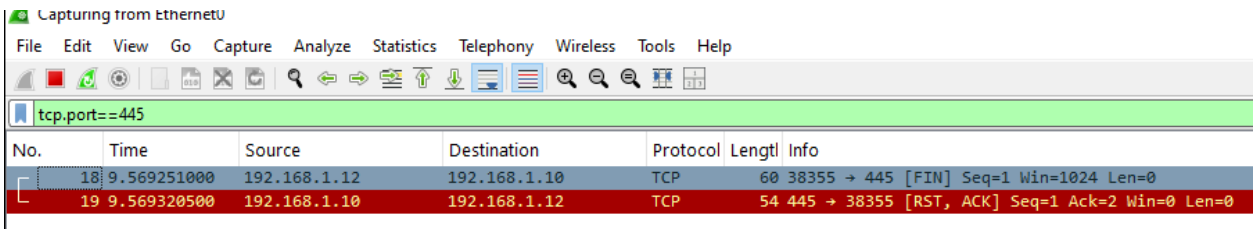
```
┌──(potato㉿kali)-[~]
└─$ sudo nmap -sN -p 445 192.168.1.10
[sudo] password for potato:
Starting Nmap 7.95 ( https://nmap.org ) at 2026-04-21 06:15 HST
Nmap scan report for 192.168.1.10
Host is up (0.00058s latency).

PORT    STATE  SERVICE
445/tcp closed microsoft-ds
MAC Address: 00:0C:29:2D:86:3F (VMware)

Nmap done: 1 IP address (1 host up) scanned in 0.13 seconds
```



c) Windows machines will always send an RST to Null, FIN, and Xmas scans, regardless of if the port is open or closed.

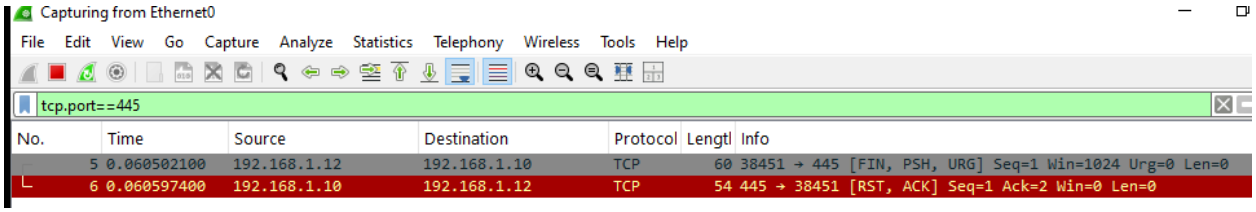
d) You can see the same result when you change the N to an F for the FIN scan:
nmap -sF -p 445 192.168.1.10

```
┌──(potato㉿kali)-[~]
└─$ sudo nmap -sF -p 445 192.168.1.10
Starting Nmap 7.95 ( https://nmap.org ) at 2026-04-21 06:19 HST
Nmap scan report for 192.168.1.10
Host is up (0.00055s latency).

PORT    STATE  SERVICE
445/tcp closed microsoft-ds
MAC Address: 00:0C:29:2D:86:3F (VMware)

Nmap done: 1 IP address (1 host up) scanned in 0.12 seconds
```

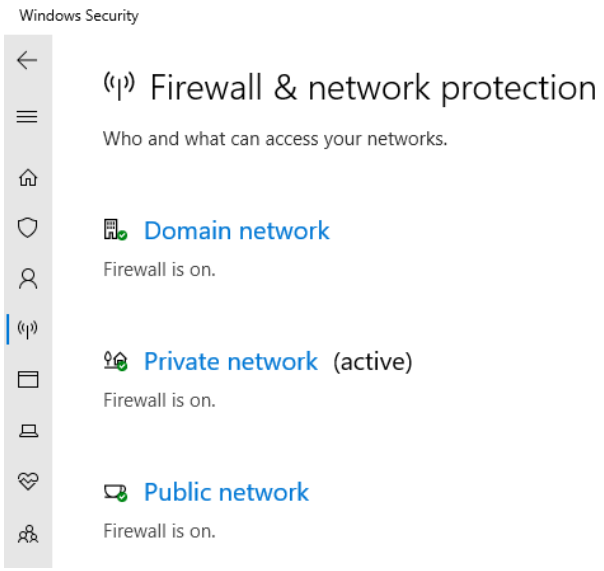


e) The same result can be seen when you use an X for the Xmas scan:
nmap -sX -p 445 192.168.1.10

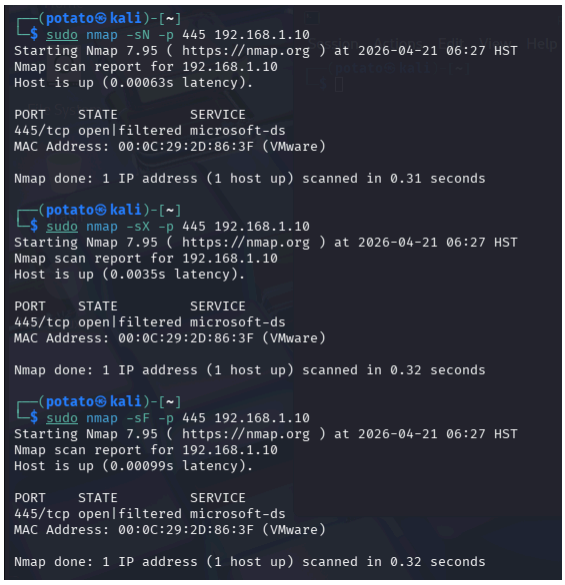
```
┌──(potato㉿kali)-[~]
└─$ sudo nmap -sX -p 445 192.168.1.10
Starting Nmap 7.95 ( https://nmap.org ) at 2026-04-21 06:21 HST
Nmap scan report for 192.168.1.10
Host is up (0.00049s latency).

PORT    STATE  SERVICE
445/tcp closed microsoft-ds
MAC Address: 00:0C:29:2D:86:3F (VMware)

Nmap done: 1 IP address (1 host up) scanned in 0.12 seconds
```

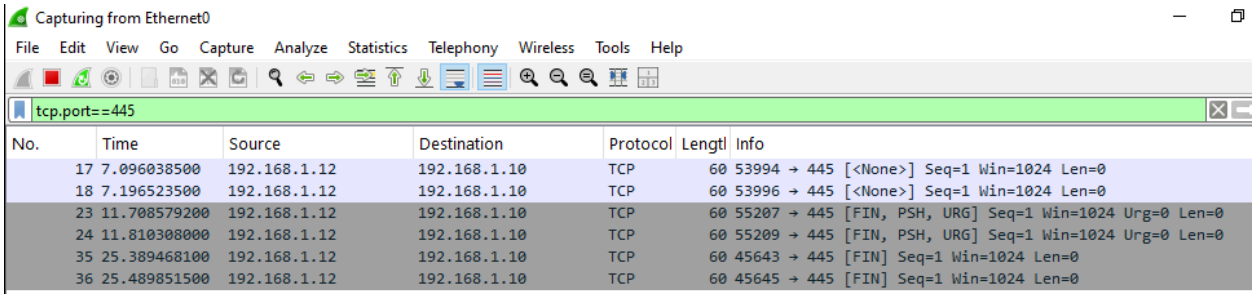


f) Turn Windows Defender Firewall back on (On WIN10 VM)

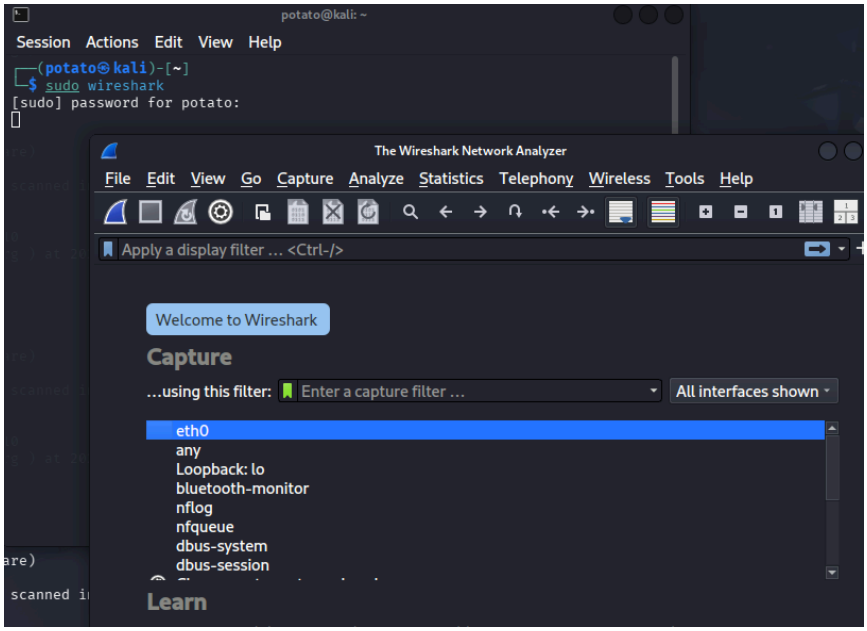


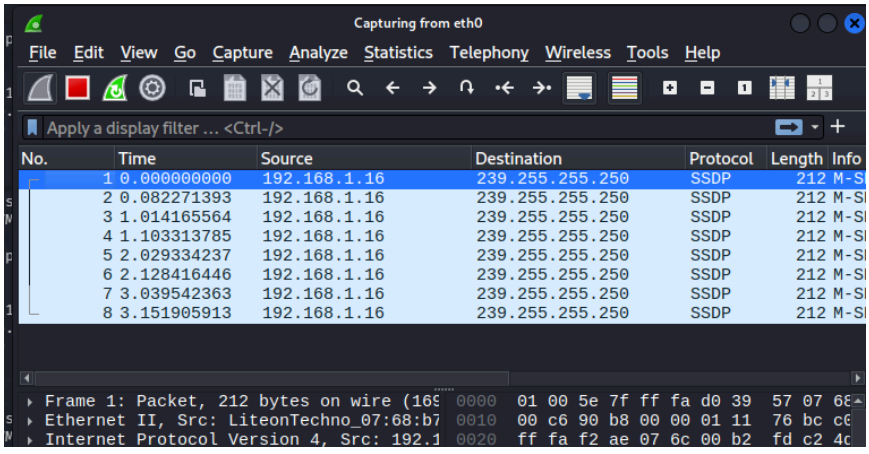
g) Execute these three scans (Null, FIN, and Xmas) again. Notice that the result in each scan has changed from closed to open or filtered. The firewall you just turned on is filtering the scans.





h) On the Kali Linux VM, open a new terminal tab. In the new terminal, start Wireshark by typing **sudo wireshark**. Double-click the eth0 interface to start sniffing.



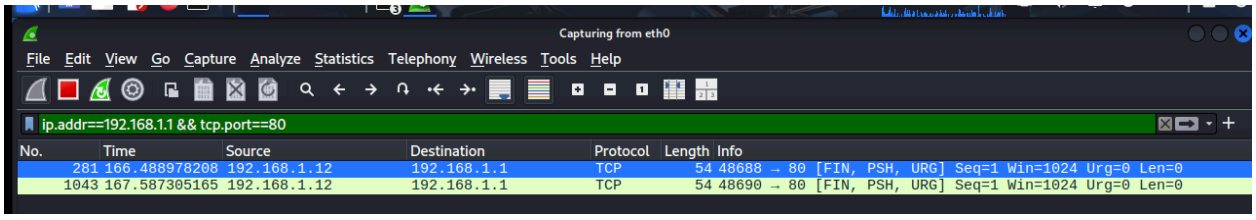


- In the original terminal tab in Kali Linux, scan your router with the Xmas scan:
sudo nmap -sX 192.168.1.1
(Substitute the IP address of your default gateway)

```
PS C:\Windows\system32> ipconfig

Windows IP Configuration


Ethernet adapter Ethernet0:

   Connection-specific DNS Suffix  . :
   IPv4 Address. . . . . . . . . . . : 192.168.1.10
   Subnet Mask . . . . . . . . . . . : 255.255.255.0
   Default Gateway . . . . . . . . . : 192.168.1.1
```

```
┌──(potato㉿kali)-[~]
└─$ sudo nmap -sX 192.168.1.1
Starting Nmap 7.95 ( https://nmap.org ) at 2026-04-21 06:38 HST
Nmap scan report for 192.168.1.1
Host is up (0.017s latency).
Not shown: 990 closed tcp ports (reset)
PORT      STATE         SERVICE
21/tcp    open|filtered ftp
22/tcp    open|filtered ssh
23/tcp    open|filtered telnet
53/tcp    open|filtered domain
80/tcp    open|filtered http
161/tcp   open|filtered snmp
443/tcp   open|filtered https
5555/tcp  open|filtered freeciv
9000/tcp  open|filtered cslistener
12345/tcp open|filtered netbus
MAC Address: D4:9A:A0:A3:84:78 (Vnpt Technology)

Nmap done: 1 IP address (1 host up) scanned in 1.60 seconds
```

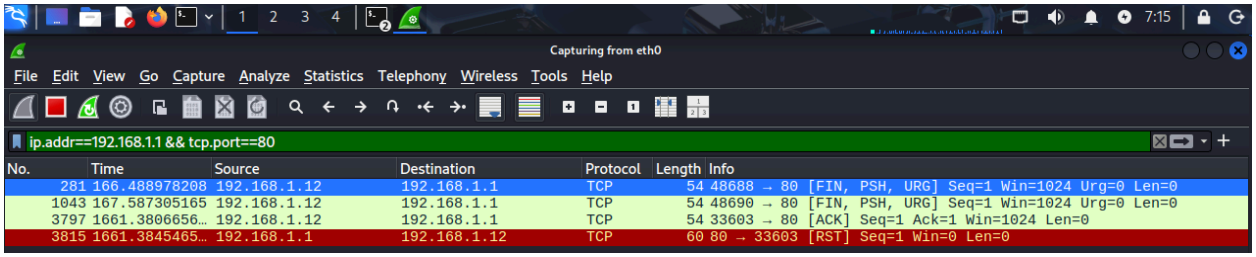
i) Keep the Wireshark capture going, and change the Wireshark display filter to **ip.addr==192.168.1.1 && tcp.port==80**



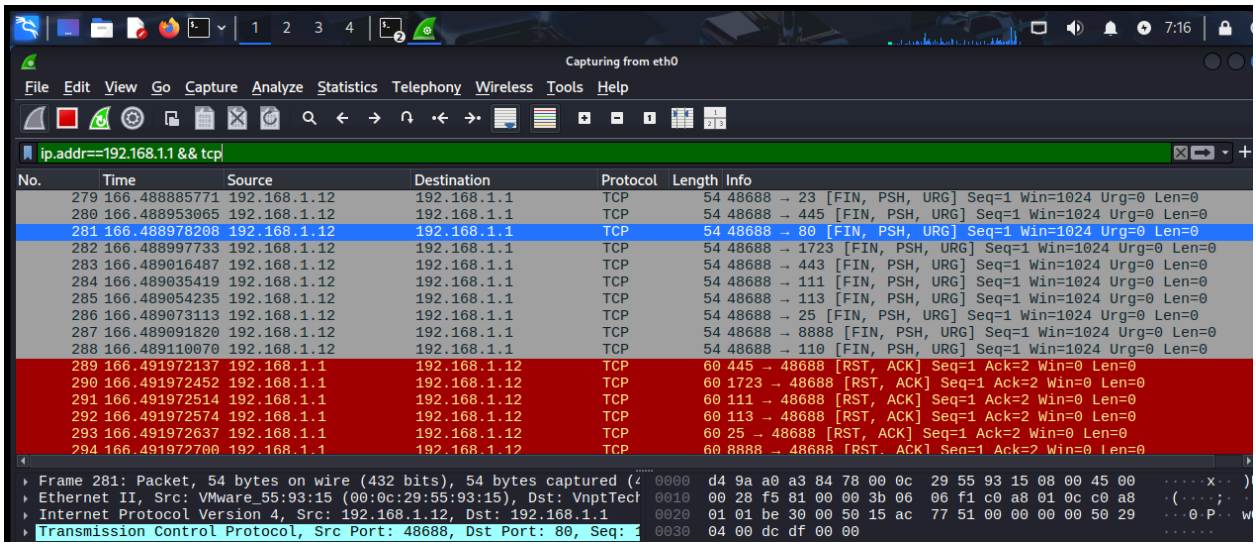
-> The scans to router on port 80 didn't return RSTs, which means either the port is open or the port is filtered.

j) The ACK scan will identify a port as filtered or unfiltered. Change to the X to an A for an ACK scan: **nmap -sA 192.168.1.1**

```
┌──(potato㉿kali)-[~]
└─$ nmap -sA 192.168.1.1
Starting Nmap 7.95 ( https://nmap.org ) at 2026-04-21 07:03 HST
Nmap scan report for 192.168.1.1
Host is up (0.0063s latency).
Not shown: 995 unfiltered tcp ports (reset)
PORT      STATE    SERVICE
22/tcp    filtered ssh
23/tcp    filtered telnet
161/tcp   filtered snmp
5555/tcp  filtered freeciv
12345/tcp filtered netbus
MAC Address: D4:9A:A0:A3:84:78 (Vnpt Technology)

Nmap done: 1 IP address (1 host up) scanned in 1.58 seconds
```

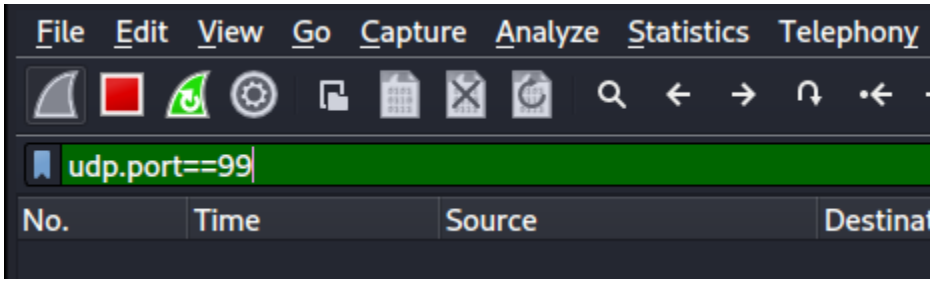


k) Change the display filter in Wireshark to the following to see all of the ACKs and RSTs. Wireshark captured them in groups of each type (the ACKs and then the RSTs). **ip.addr==192.168.1.1 && tcp**

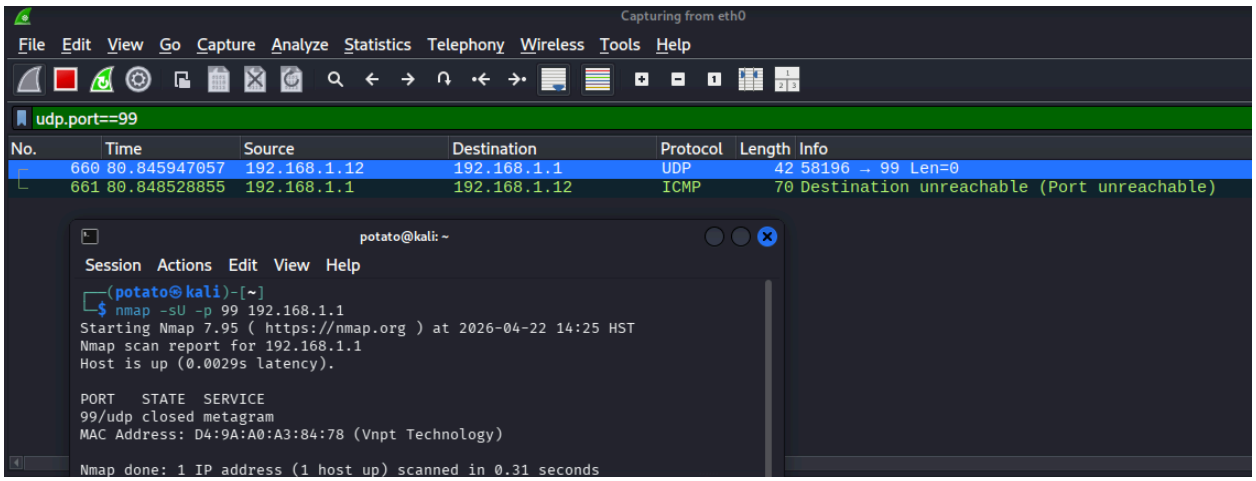


4a-4e

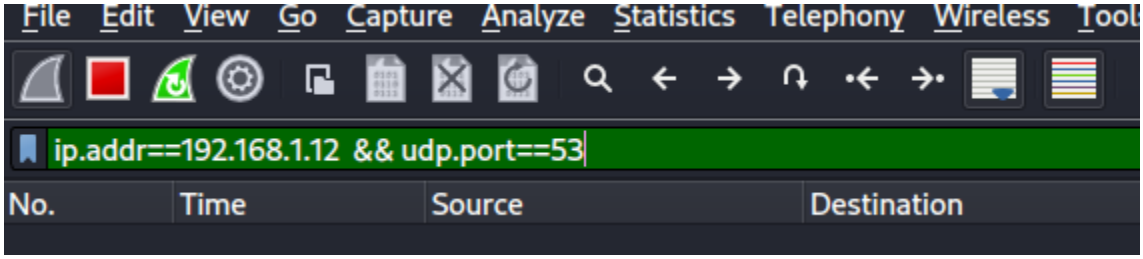
a) On the Kali Linux VM, start a new Wireshark capture. Use a display filter of **udp.port==99**



b) First, send a UDP scan to **port 99** of the router: **nmap -sU -p 99 192.168.1.1**

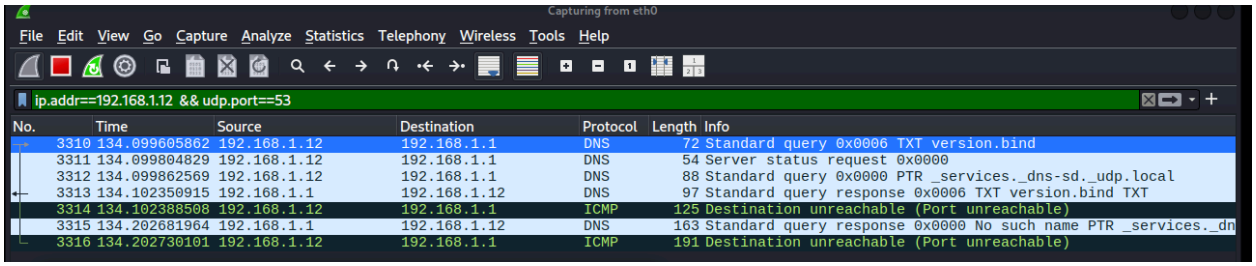


c) Start a new capture and filter by the IP address of the Kali Linux VM: **ip.addr==192.168.1.12 && udp.port==53**

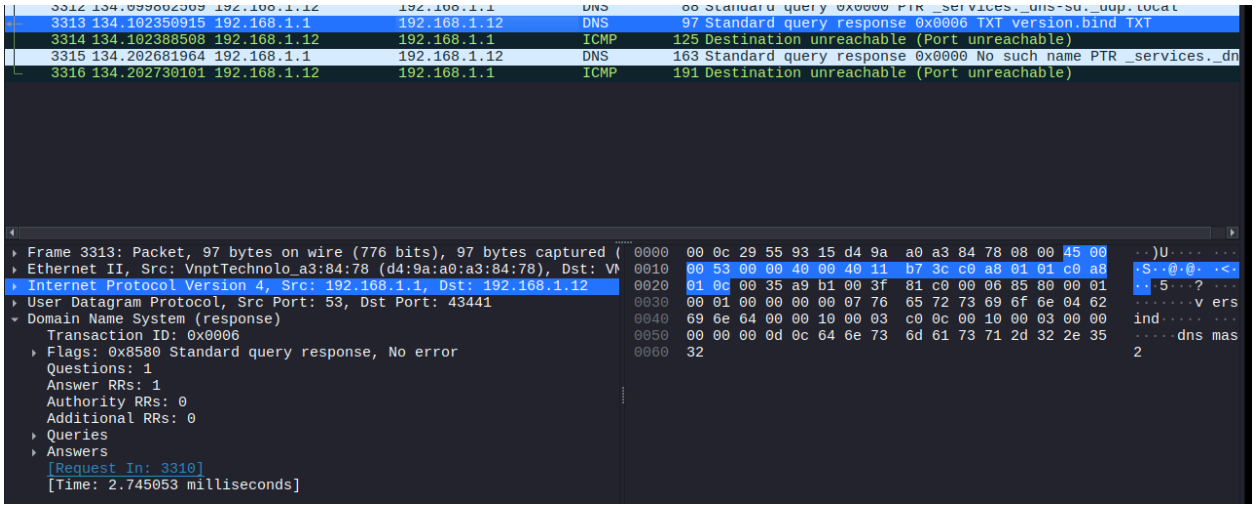


d) In Kali Linux, probe for a service that uses UDP listening on port 53 of that box that everyone simply calls router: **sudo nmap -sU -p 53 192.168.1.1**

```
┌──(potato㉿kali)-[~]
└─$ sudo nmap -sU -p 53 192.168.1.1
[sudo] password for potato:
Starting Nmap 7.95 ( https://nmap.org ) at 2026-04-22 14:28 HST
Nmap scan report for 192.168.1.1
Host is up (0.0064s latency).

PORT   STATE SERVICE
53/udp open  domain
MAC Address: D4:9A:A0:A3:84:78 (Vnpt Technology)

Nmap done: 1 IP address (1 host up) scanned in 0.20 seconds
```

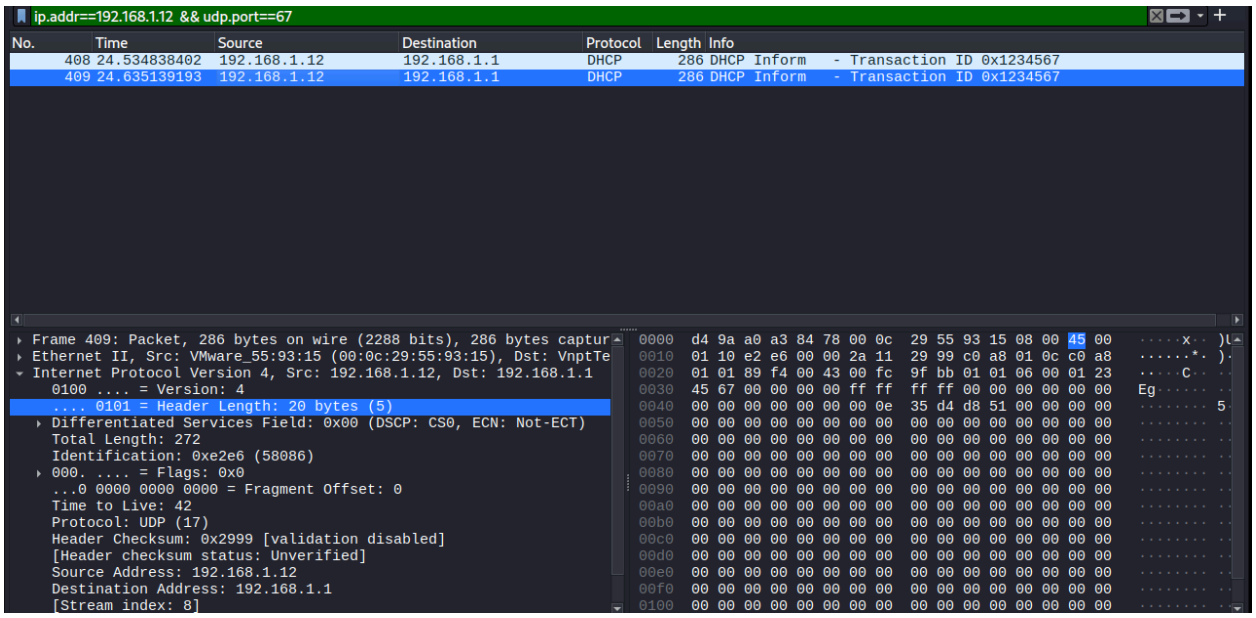


- DNS system response, evidence of a DNS server:



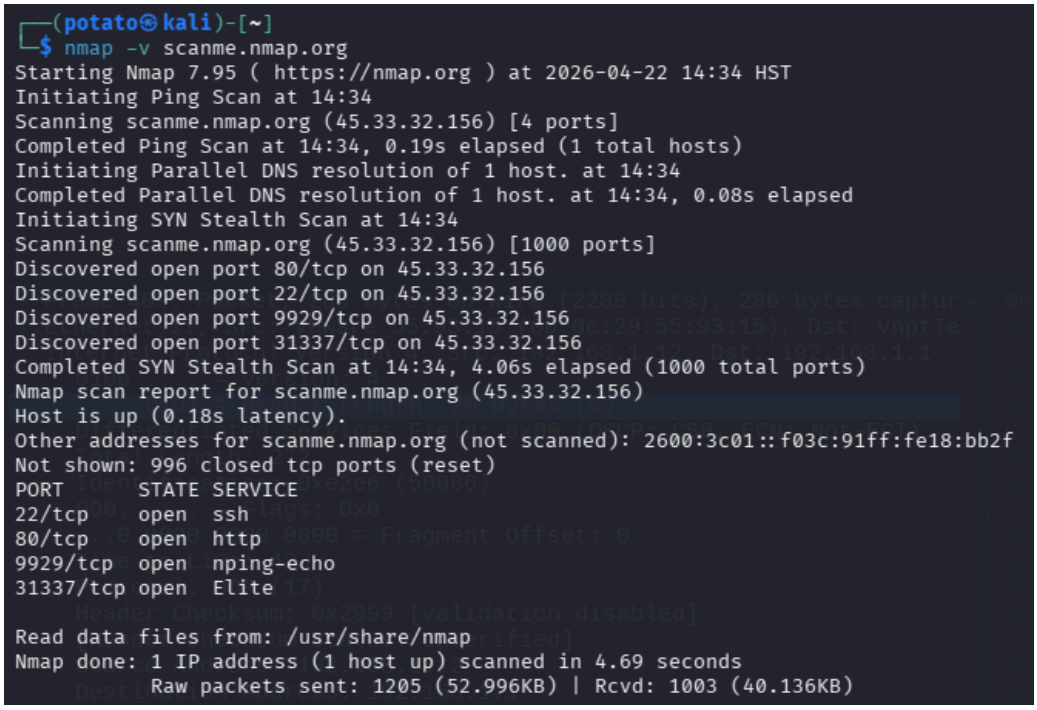
e) Now try a UDP scan with port 67: **nmap -sU -p 67 192.168.1.1**

```
┌──(potato㉿kali)-[~]
└─$ nmap -sU -p 67 192.168.1.1
Starting Nmap 7.95 ( https://nmap.org ) at 2026-04-22 14:31 HST
Nmap scan report for 192.168.1.1
Host is up (0.0019s latency).

PORT   STATE         SERVICE
67/udp open|filtered dhcps
MAC Address: D4:9A:A0:A3:84:78 (Vnpt Technology)

Nmap done: 1 IP address (1 host up) scanned in 0.36 seconds
```



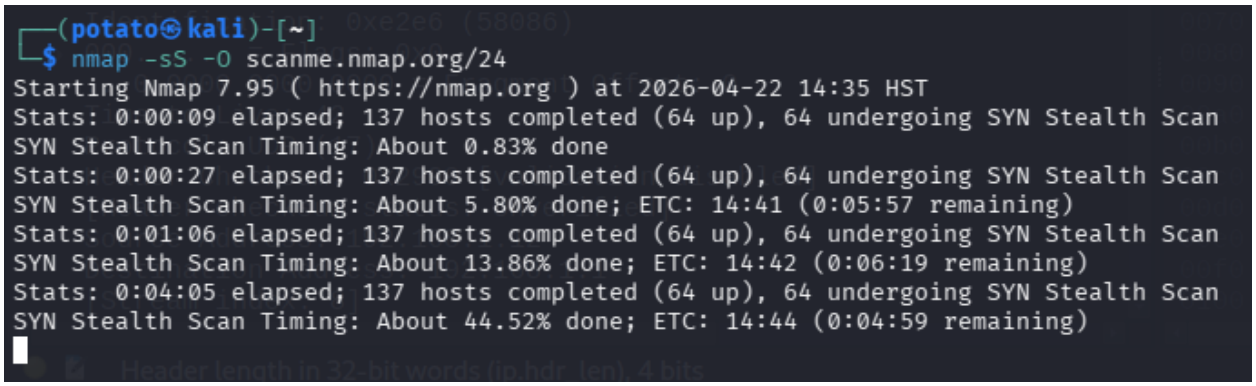
5a-5b

a) Execute the following command: **nmap -v scanme.nmap.org**



b) Execute the following command: **nmap -sS -O scanme.nmap.org/24**

This launches a stealth SYN scan against each machine that is up out of the 256 IPs on the Class C–sized network where Scanme resides. That's the reason why this scan takes much longer than other ordinary scans, and there is a lot of obtained information.



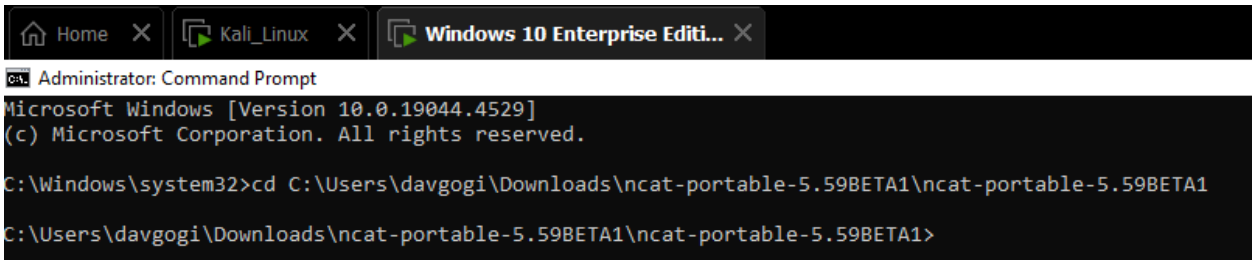
```
Stats: 0:06:47 elapsed; 137 hosts completed (64 up), 64 undergoing SYN Stealth Scan
SYN Stealth Scan Timing: About 72.92% done; ETC: 14:44 (0:02:29 remaining)
Stats: 0:09:09 elapsed; 137 hosts completed (64 up), 64 undergoing SYN Stealth Scan
SYN Stealth Scan Timing: About 95.25% done; ETC: 14:44 (0:00:27 remaining)
Nmap scan report for mail.ai-helper.asia (45.33.32.4)
Host is up (0.18s latency).
Not shown: 993 filtered tcp ports (no-response)
PORT    STATE  SERVICE
22/tcp  open   ssh
25/tcp  open   smtp
80/tcp  open   http
443/tcp open   https
587/tcp closed submission
993/tcp open   imaps
995/tcp closed pop3s
Aggressive OS guesses: Linux 5.0 - 5.14 (98%), MikroTik RouterOS 7.2 - 7.5 (Linux 5.6.3) (97%), Linux 4.15 - 5.19 (94%), Linux 2.6.32 - 3.13 (93%), OpenWrt
21.02 (Linux 5.4) (93%), Linux 2.6.39 (93%), Linux 5.1 - 5.15 (92%), Linux 6.0 (92%), OpenWrt 22.03 (Linux 5.10) (92%), Linux 2.6.22 - 2.6.36 (91%)
No exact OS matches for host (test conditions non-ideal).

Nmap scan report for 45-33-32-5.ip.linodeusercontent.com (45.33.32.5)
Host is up (0.18s latency).
Not shown: 995 filtered tcp ports (no-response)
PORT     STATE  SERVICE
22/tcp   open   ssh
80/tcp   open   http
443/tcp  closed https
8000/tcp closed http-alt
8080/tcp open   http-proxy
Device type: general purpose|router|storage-misc
Running (JUST GUESSING): Linux 5.X|4.X|2.6.X|3.X (98%), MikroTik RouterOS 7.X (96%), HP embedded (89%)
OS CPE: cpe:/o:linux:linux_kernel:5 cpe:/o:mikrotik:routeros:7 cpe:/o:linux:linux_kernel:5.6.3 cpe:/o:linux:linux_kernel:4 cpe:/o:linux:linux_kernel:2.6 cp
:/o:linux:linux_kernel:3 cpe:/o:linux:linux_kernel:6.0 cpe:/h:hp:p2000_g3
Aggressive OS guesses: Linux 5.0 - 5.14 (98%), MikroTik RouterOS 7.2 - 7.5 (Linux 5.6.3) (96%), Linux 4.15 - 5.19 (94%), Linux 2.6.32 - 3.13 (93%), Linux 2
6.39 (93%), OpenWrt 21.02 (Linux 5.4) (92%), Linux 6.0 (92%), OpenWrt 22.03 (Linux 5.10) (92%), Linux 2.6.22 - 2.6.36 (91%), Linux 5.1 - 5.15 (91%)
No exact OS matches for host (test conditions non-ideal).

Nmap scan report for li982-9.members.linode.com (45.33.32.9)
Host is up (0.18s latency).
Not shown: 998 closed tcp ports (reset)
PORT    STATE SERVICE
80/tcp  open  http
443/tcp open  https
Aggressive OS guesses: Linux 5.0 - 5.14 (97%), Linux 4.15 - 5.19 (97%), MikroTik RouterOS 7.2 - 7.5 (Linux 5.6.3) (96%), Linux 2.6.32 - 3.13 (94%), OpenWrt
```

Lab Exercise 16.02: Sockets with netcat (nc) and ncat

Step 1: Create a simple chat server with netcat/ncat.

1a-1f

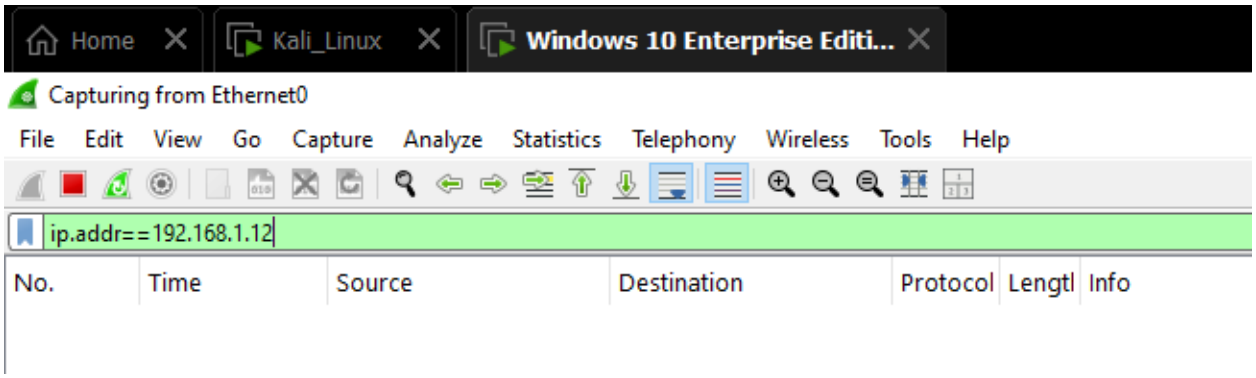
a) On the Windows 10 VM that will act as the netcat server, start listening, in the location you changed directories to, on port 52000 with the following command:
ncat -lp 52000



- Start listening on port 52000 with ncat from the WIN10 VM:

```
C:\Users\davgogi\Downloads\ncat-portable-5.59BETA1\ncat-portable-5.59BETA1>ncat -lp 52000
```

b) Filtering packets from Kali Linux's machine IP address on WIN10 VM's Wireshark: **ip.addr==192.168.1.12**

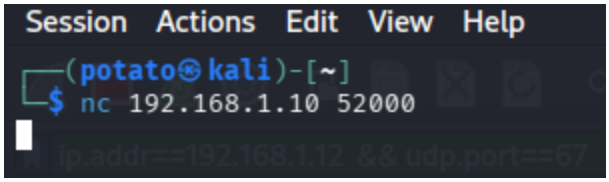


c) Verify that the port is open with another utility with a similar sounding name, netstat. **netstat -an | more**

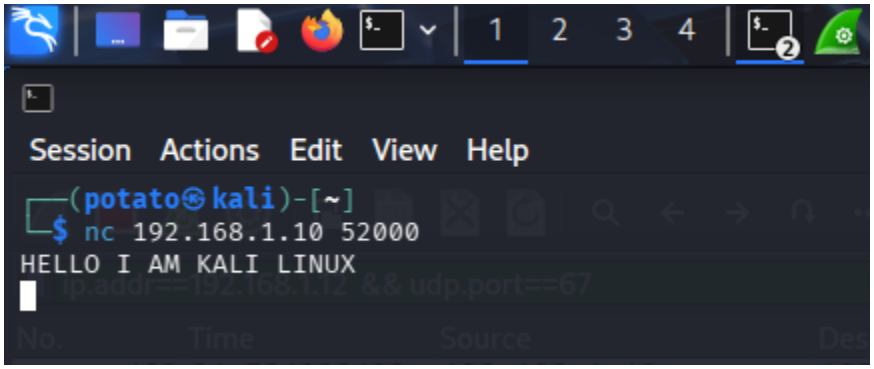
```
Active Connections

  Proto  Local Address          Foreign Address        State
  TCP    0.0.0.0:135            0.0.0.0:0              LISTENING
  TCP    0.0.0.0:445            0.0.0.0:0              LISTENING
  TCP    0.0.0.0:5040           0.0.0.0:0              LISTENING
  TCP    0.0.0.0:5357           0.0.0.0:0              LISTENING
  TCP    0.0.0.0:7680           0.0.0.0:0              LISTENING
  TCP    0.0.0.0:49664          0.0.0.0:0              LISTENING
  TCP    0.0.0.0:49665          0.0.0.0:0              LISTENING
  TCP    0.0.0.0:49666          0.0.0.0:0              LISTENING
  TCP    0.0.0.0:49667          0.0.0.0:0              LISTENING
  TCP    0.0.0.0:49668          0.0.0.0:0              LISTENING
  TCP    0.0.0.0:49669          0.0.0.0:0              LISTENING
  TCP    0.0.0.0:49670          0.0.0.0:0              LISTENING
  TCP    0.0.0.0:50032          0.0.0.0:0              LISTENING
  TCP    0.0.0.0:52000          0.0.0.0:0              LISTENING
  TCP    192.168.1.10:139       0.0.0.0:0              LISTENING
  TCP    192.168.1.10:49744     4.213.25.240:443       ESTABLISHED
```

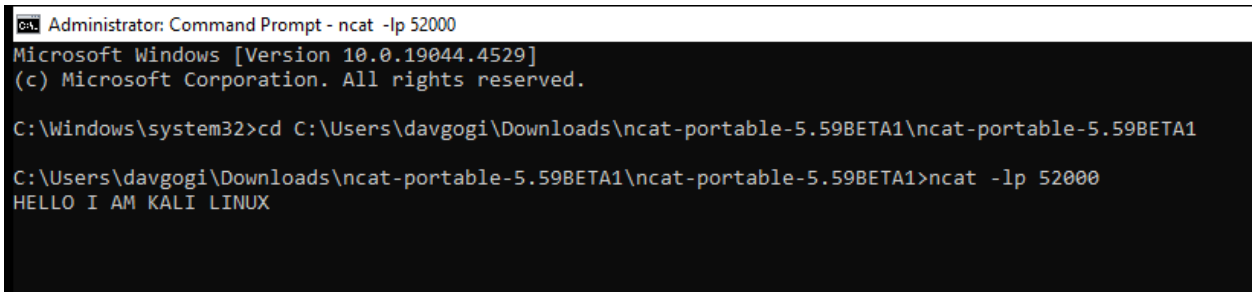
- We can see that port 52000 is listening, which means ncat is working.

d) On the Kali Linux VM acting as the netcat client, type the following: **nc <IP server of WIN10 VM running netcat> 52000 -> nc 192.168.1.10 52000**



e) Start typing in each machine. The messages will appear in both machine.

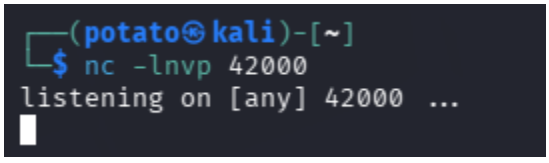




f) Break out of the connection by pressing CTRL-C on either machine

e) Reverse roles by making the Kali Linux VM the netcat server and the Windows 10 machine the netcat client.

- Start listening on port 42000 from the Kali Linux VM with ip 192.168.1.12:
nc -lnvp 42000



- Connect to Kali Linux VM from WIN10 VM with ncat: **ncat 192.168.1.12 42000**

```
C:\Users\davgogi\Downloads\ncat-portable-5.59BETA1\ncat-portable-5.59BETA1>ncat 192.168.1.12 42000
HELLO IM WIN10 VM
```

```
┌──(potato㉿kali)-[~]
└─$ nc -lnvp 42000
listening on [any] 42000 ...
connect to [192.168.1.12] from (UNKNOWN) [192.168.1.10] 50122
HELLO IM WIN10 VM
```

2a-2e:

a) On the Windows 10 VM, open a port. Using the output redirection operator (>), specify that whatever comes through port 55555 doesn't get output in the console, like before, but rather goes into a file called alice.txt, which doesn't exist just yet.
ncat -lp 55555 > alice.txt

```
C:\Users\davgogi\Downloads\ncat-portable-5.59BETA1\ncat-portable-5.59BETA1>ncat -lp 55555 > alice.txt
```

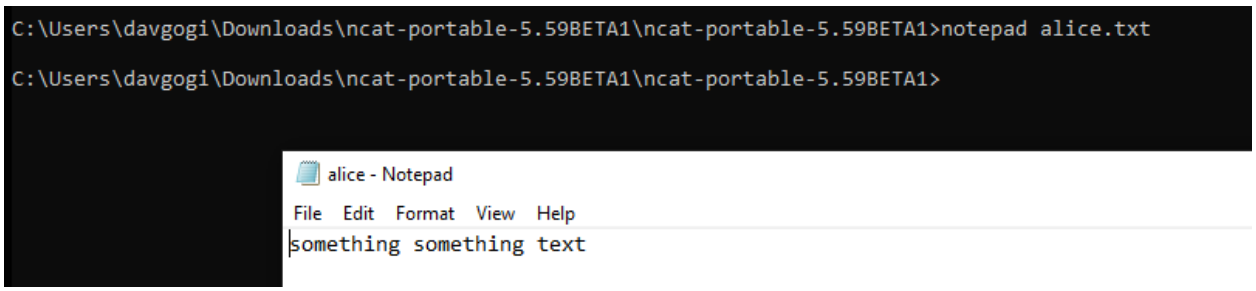
b) On the Kali Linux VM, create a text file, put some text in the file, and then save it as bob.txt and close the file.

```
┌──(potato㉿kali)-[~]
└─$ echo "something something text" > bob.txt
```

c) On the Kali Linux VM, from the same directory you've been in, execute the following command (with the IP address of the Windows 10 VM):
nc -w 1 192.168.1.10 55555 < bob.txt

```
┌──(potato㉿kali)-[~]
└─$ nc -w 1 192.168.1.10 55555 < bob.txt

┌──(potato㉿kali)-[~]
└─$
```

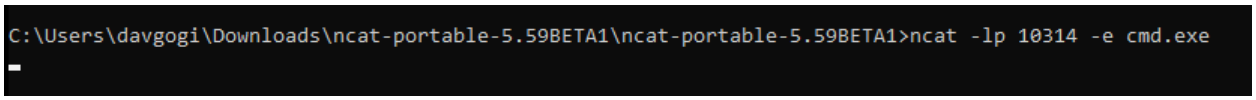
d) On the Windows 10 VM, type notepad alice.txt to see the contents of the source's bob.txt file in the target's alice.txt file in Notepad.



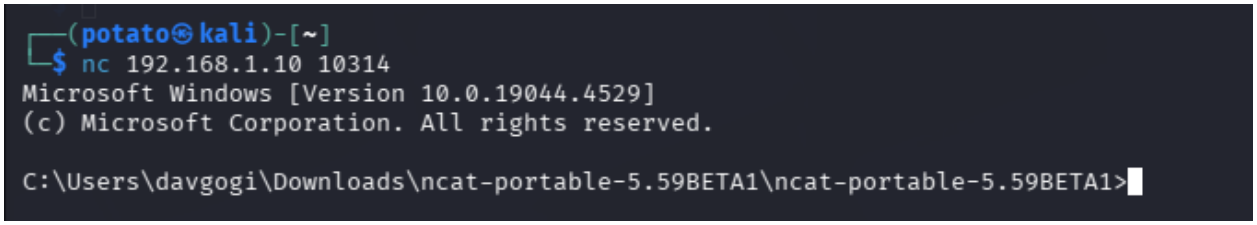
e) Same thing but reverse roles.

3a-3e:

a) On the Windows 10 VM, type the following command: **ncat -lp 10314 -e cmd.exe**



b) On the Kali Linux VM, type the following command: **nc 192.168.1.10 10314**



c) Any Windows command you type will now be sent to the victim machine and executed on that machine. Try these:
ipconfig /all
arp -a
route print

```
┌──(potato㉿kali)-[~]
└─$ nc 192.168.1.10 10314
Microsoft Windows [Version 10.0.19044.4529]
(c) Microsoft Corporation. All rights reserved.

C:\Users\davgogi\Downloads\ncat-portable-5.59BETA1\ncat-portable-5.59BETA1>ipconfig /all
ipconfig /all

Windows IP Configuration

   Host Name . . . . . . . . . . . . : WIN10-PC
   Primary Dns Suffix  . . . . . . . : corpo.lab
   Node Type . . . . . . . . . . . . : Hybrid
   IP Routing Enabled. . . . . . . . : No
   WINS Proxy Enabled. . . . . . . . : No
   DNS Suffix Search List. . . . . . : corpo.lab

Ethernet adapter Ethernet0:

   Connection-specific DNS Suffix  . :
   Description . . . . . . . . . . . : Intel(R) 82574L Gigabit Network Connection
   Physical Address. . . . . . . . . : 00-0C-29-2D-86-3F
   DHCP Enabled. . . . . . . . . . . : No
   Autoconfiguration Enabled . . . . : Yes
   IPv4 Address. . . . . . . . . . . : 192.168.1.10(Preferred)
   Subnet Mask . . . . . . . . . . . : 255.255.255.0
   Default Gateway . . . . . . . . . : 192.168.1.1
   DNS Servers . . . . . . . . . . . : 192.168.1.4
                                       8.8.8.8
   NetBIOS over Tcpip. . . . . . . . : Enabled
```

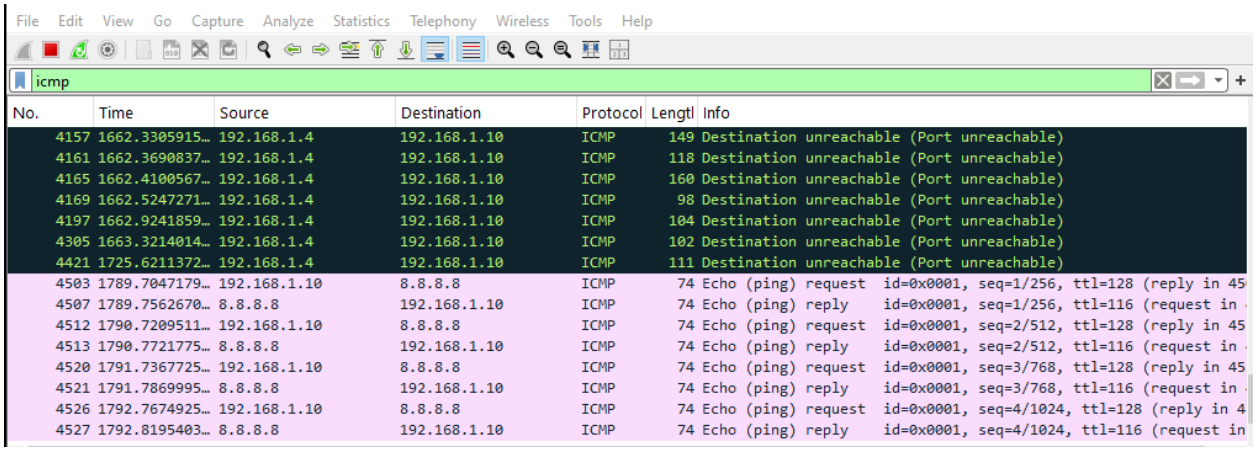
```
C:\Users\davgogi\Downloads\ncat-portable-5.59BETA1\ncat-portable-5.59BETA1>arp -a
arp -a

Interface: 192.168.1.10 --- 0x9
  Internet Address      Physical Address      Type
  192.168.1.1           d4-9a-a0-a3-84-78     dynamic
  192.168.1.4           30-1b-97-36-52-0d     dynamic
  192.168.1.12          00-0c-29-55-93-15     dynamic
  192.168.1.16          d0-39-57-07-68-b7     dynamic
  192.168.1.255         ff-ff-ff-ff-ff-ff     static
  224.0.0.2             01-00-5e-00-00-02     static
  224.0.0.22            01-00-5e-00-00-16     static
  224.0.0.251           01-00-5e-00-00-fb     static
  224.0.0.252           01-00-5e-00-00-fc     static
  239.255.255.250       01-00-5e-7f-ff-fa     static
```

```
C:\Users\davgogi\Downloads\ncat-portable-5.59BETA1\ncat-portable-5.59BETA1>route print
route print
===========================================================================
Interface List
  9...00 0c 29 2d 86 3f ......Intel(R) 82574L Gigabit Network Connection
  1...........................Software Loopback Interface 1
===========================================================================

IPv4 Route Table
===========================================================================
Active Routes:
Network Destination        Netmask          Gateway       Interface  Metric
          0.0.0.0          0.0.0.0      192.168.1.1     192.168.1.10    281
        127.0.0.0        255.0.0.0         On-link         127.0.0.1    331
        127.0.0.1  255.255.255.255         On-link         127.0.0.1    331
  127.255.255.255  255.255.255.255         On-link         127.0.0.1    331
      192.168.1.0    255.255.255.0         On-link      192.168.1.10    281
     192.168.1.10  255.255.255.255         On-link      192.168.1.10    281
    192.168.1.255  255.255.255.255         On-link      192.168.1.10    281
        224.0.0.0        240.0.0.0         On-link         127.0.0.1    331
        224.0.0.0        240.0.0.0         On-link      192.168.1.10    281
  255.255.255.255  255.255.255.255         On-link         127.0.0.1    331
  255.255.255.255  255.255.255.255         On-link      192.168.1.10    281
===========================================================================
Persistent Routes:
  Network Address          Netmask  Gateway Address  Metric
          0.0.0.0          0.0.0.0      192.168.1.1  Default
===========================================================================

IPv6 Route Table
===========================================================================
Active Routes:
 If Metric Network Destination      Gateway
  1    331 ::1/128                  On-link
  1    331 ff00::/8                 On-link
===========================================================================
Persistent Routes:
  None
```

d) You can even sniff all packets in Wireshark containing all commands and their corresponding output.

- Filter by **icmp** in Wireshark on the Windows 10 VM.
- From the Windows command prompt on the Kali Linux VM, enter the following:
ping 8.8.8.8

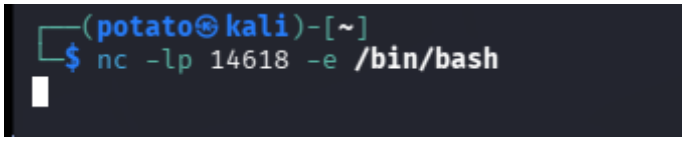
```
C:\Users\davgogi\Downloads\ncat-portable-5.59BETA1\ncat-portable-5.59BETA1>ping 8.8.8.8
ping 8.8.8.8

Pinging 8.8.8.8 with 32 bytes of data:
Reply from 8.8.8.8: bytes=32 time=51ms TTL=116
Reply from 8.8.8.8: bytes=32 time=51ms TTL=116
Reply from 8.8.8.8: bytes=32 time=50ms TTL=116
Reply from 8.8.8.8: bytes=32 time=52ms TTL=116

Ping statistics for 8.8.8.8:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
Approximate round trip times in milli-seconds:
    Minimum = 50ms, Maximum = 52ms, Average = 51ms
```



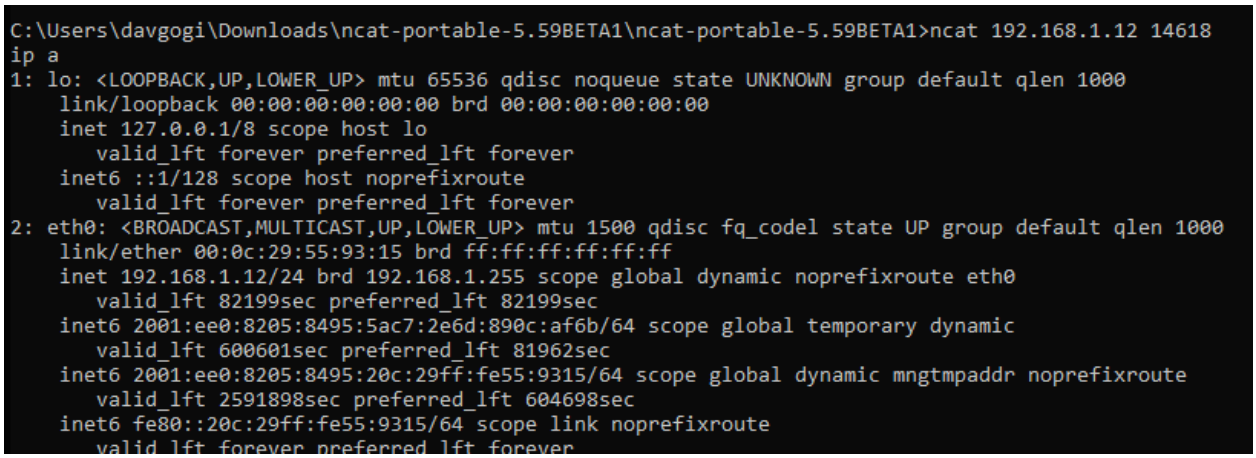
This ncat shell creation can even be made persistent by adding it to the victim machine's Registry in one of the following locations:
-HKEY_LOCAL_MACHINE\Software\Microsoft\Windows\Curre ntVersion\Run
-HKEY_CURRENT_USER\Software\Microsoft\Windows\Current Version\Run
-HKEY_LOCAL_MACHINE\Software\Microsoft\Windows\Curre ntVersion\RunOnce
-HKEY_CURRENT_USER\Software\Microsoft\Windows\Current Version\RunOnce

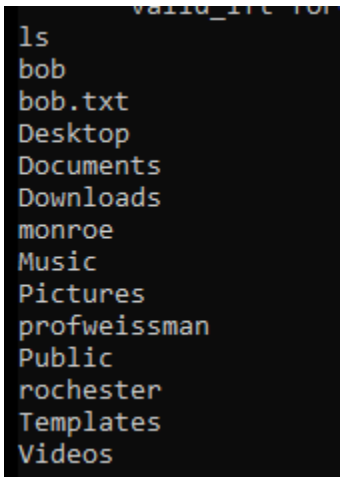
e) Reverse roles and use Bash from the Kali Linux VM from the Windows 10 VM.

- On the Kali Linux VM, type the following command: **nc -lp 14618 -e /bin/bash**



- On the Windows 10 VM, type the following command: **ncat 192.168.1.12 14618**





f) Filter by ICMP in Wireshark on the Kali Linux VM:

- From the Windows command prompt on the Kali Linux VM, enter the following:
ping 1.1.1.1

```
ping 1.1.1.1
PING 1.1.1.1 (1.1.1.1) 56(84) bytes of data.
64 bytes from 1.1.1.1: icmp_seq=1 ttl=56 time=77.4 ms
64 bytes from 1.1.1.1: icmp_seq=2 ttl=56 time=41.6 ms
64 bytes from 1.1.1.1: icmp_seq=3 ttl=56 time=40.6 ms
64 bytes from 1.1.1.1: icmp_seq=4 ttl=56 time=41.0 ms
64 bytes from 1.1.1.1: icmp_seq=5 ttl=56 time=41.9 ms
64 bytes from 1.1.1.1: icmp_seq=6 ttl=56 time=42.8 ms
```

- Now you're actually sending a ping from the Kali Linux VM through the Windows 10 VM. The replies will be sent to the Kali Linux VM: